BMad Harness Extension

> A verification, permission, state, observability, and tool-governance extension for BMad Method.

1. Project Overview

BMad Method is excellent for AI-assisted software development planning. It helps teams move from idea to PRD, architecture, epics, stories, and agentic implementation.

However, when AI agents move from planning to execution, teams need stronger governance:

- Did the agent actually complete the task?
- Were tests really executed?
- Did the agent modify files outside the story scope?
- Did the agent touch security-sensitive logic?
- Which tools and commands were used?
- What failed, and how was it fixed?
- Is there an auditable trace for human review?

BMad Harness Extension adds the missing execution governance layer.

In one sentence:

> BMad solves planning. BMad Harness solves controlled execution.

中文理解：

> BMad 解决“AI 如何参与软件开发流程”；BMad Harness 解决“AI 如何被安全、可验证、可审计地执行”。

---

2. Core Philosophy

AI coding should not be treated as a black-box conversation.

A production-ready AI development workflow should be:

```
Planned
→ Scoped
→ Permission-aware
→ Tool-governed
→ State-tracked
→ Verified
→ Reviewed
```

```
→ Logged
→ Auditable
```

BMad Harness Extension introduces five execution governance layers:

```
V - Verification
S - State
P - Permission
O - Observability
T - Tool Governance
```

Together, these form the **VSPOT Harness**.

---

3. What This Extension Adds

BMad Method already provides:

```
Idea
→ PRD
→ Architecture
→ Epics
→ Stories
→ Dev Agent
→ QA / Review
```

BMad Harness Extension adds:

```
Story Readiness Check
→ Permission Check
→ Tool Policy Check
→ Dev Execution Protocol
→ State Tracking
→ Verification Gates
→ Failure Attribution
→ Human Review
→ Execution Log
```

Final integrated workflow:

```
Idea
→ PRD
→ Architecture
→ Epics
→ Stories
```

```
→ Story Readiness Check
→ Permission Check
→ Dev Execution
→ Test Execution
→ Verification Gate
→ Review Gate
→ Trace Log
→ Done
```

---

4. Project Goals

This project aims to provide a reusable template and operating model for teams using BMad Method, Claude Code, Cursor, Windsurf, GitHub Copilot, or other AI coding agents.

The goal is not to replace BMad.

The goal is to extend BMad from:

```
AI-assisted planning and implementation
```

to:

```
AI-governed, verifiable, auditable software execution
```

---

5. Target Users

This project is designed for:

- AI coding users
- BMad Method users
- AI-native product teams
- software teams using coding agents
- enterprise AI implementation consultants
- teams building digital workers or agentic workflows
- founders building AI-assisted engineering systems

---

6. Core Concepts

6.1 Harness

A harness is the execution environment and governance layer around an AI agent.

It defines:

- what the agent can do
- what the agent cannot do
- which tools the agent can use
- how progress is tracked
- how results are verified
- how failures are handled
- how human approval is triggered
- how execution is logged

6.2 Verification

AI completion is not accepted by statement.

Completion must be verified by objective gates:

```
Tests passed
Build passed
Lint passed
Acceptance criteria met
Scope respected
Risk reviewed
Execution logged
```

6.3 State

AI agents need explicit state.

Without state tracking, agents may:

- repeat work
- skip steps
- lose context
- claim completion too early
- fail to recover from errors

6.4 Permission

AI agents must operate within clear boundaries.

Some actions are safe.

Some require human approval.

Some are forbidden.

6.5 Observability

Every meaningful agent action should leave a trace:

- input documents
- files inspected
- files changed
- tools used
- commands run
- failures encountered
- fixes applied
- final verification result

6.6 Tool Governance

AI agents should not use tools freely without policy.

Tool governance defines:

- allowed read tools
- allowed write tools
- allowed commands
- approval-required commands
- forbidden commands

---

7. Recommended Repository Structure

```
bmad-harness-extension/
  README.md
  LICENSE
  docs/
    philosophy.md
    bmad-integration.md
    enterprise-use-cases.md
    vspot-framework.md
    failure-taxonomy.md
  templates/
    .ai-harness/
      harness-manifest.md
      permissions.md
      tool-policy.md
      story-state.json
      verification-gates.md
      execution-log.md
      failure-taxonomy.md
      human-approval-policy.md
  commands/
    dev-agent-protocol.md
    qa-agent-protocol.md
```

```
    story-readiness-check.md
    review-gate.md
    verification-gate.md
    permission-check.md
  examples/
    password-reset/
      README.md
      .ai-harness/
        harness-manifest.md
        permissions.md
        tool-policy.md
        story-state.json
        verification-gates.md
        execution-log.md
    hr-screening-agent/
      README.md
      .ai-harness/
        harness-manifest.md
        permissions.md
        tool-policy.md
        story-state.json
        verification-gates.md
        execution-log.md
  checklists/
    story-readiness-checklist.md
    verification-checklist.md
    permission-risk-checklist.md
    release-readiness-checklist.md
```

---

8. MVP Scope

The first version should be simple.

Do not build a CLI first.

Do not over-engineer.

Start with reusable Markdown templates and protocols.

MVP should include:

```
.ai-harness/
  harness-manifest.md
  permissions.md
  tool-policy.md
  story-state.json
```

```
  verification-gates.md
  execution-log.md
```

And three agent protocols:

```
commands/
  dev-agent-protocol.md
  qa-agent-protocol.md
  human-review-protocol.md
```

MVP success definition:

```
A BMad user can copy this template into their repo
and immediately use it to govern AI coding execution.
```

---

9. Core Template Files

9.1 `.ai-harness/harness-manifest.md`

```
# AI Harness Manifest

## Project Name

[Project Name]

## Project Goal

Describe what this system is supposed to achieve.

## Harness Goal

This harness exists to make AI-assisted development:

- controlled
- verifiable
- auditable
- permission-aware
- human-reviewable

## AI Execution Principle

AI agents may assist with:

- planning
- coding
```

```
- testing
- documentation
- refactoring within approved scope

AI agents must not bypass:

- tests
- security checks
- human approvals
- permission boundaries
- verification gates

## Done Means

A story is done only when:

- acceptance criteria are met
- required tests pass
- build passes
- lint/typecheck pass
- changed files are within story scope
- execution log is complete
- human review is completed if required

## Agent Roles

### PM Agent

Responsible for:

- requirements clarification
- PRD generation
- user story definition
- business acceptance criteria

### Architect Agent

Responsible for:

- system design
- architecture documentation
- technical constraints
- integration strategy

### Scrum Master / Story Agent

Responsible for:

- story breakdown
- story readiness
- dependency identification
```

```
- execution sequence

### Dev Agent

Responsible for:

- implementation
- unit tests
- local verification
- execution logging

### QA Agent

Responsible for:

- test review
- acceptance verification
- regression risk identification
- quality gate enforcement

### Human Owner

Responsible for:

- approval of high-risk actions
- final review when needed
- business-level judgment
- production release approval

## Default Rule

If the agent is uncertain whether an action is allowed, it must stop and
request human approval.
```

9.2 `.ai-harness/permissions.md`

```
# AI Agent Permission Policy

## Purpose

This file defines what AI agents are allowed to do, what requires human
approval, and what is forbidden.

## Allowed Without Approval

AI agents may perform the following actions without additional approval:

- read project files
```

```
- search the codebase
- inspect git diff
- inspect local logs
- create new files within the current story scope
- edit files within the current story scope
- create or update test files related to the current story
- update documentation related to the current story
- run local verification commands
- run lint commands
- run typecheck commands
- run unit tests
- run integration tests
- run build commands

## Requires Human Approval

AI agents must request human approval before performing the following
actions:

- delete files
- rename many files
- modify database schema
- modify authentication logic
- modify authorization logic
- modify payment or billing logic
- modify production configuration
- modify deployment configuration
- add new third-party dependencies
- update major dependency versions
- touch more than 10 files
- refactor unrelated modules
- change public API contracts
- change data migration logic
- change encryption or security-sensitive logic
- access external services
- send emails or notifications
- push commits to remote repository
- deploy to any environment

## Forbidden Actions

AI agents must never perform the following actions:

- access production secrets
- expose private keys
- modify billing data directly
- bypass tests
- disable security checks
- remove audit logs
- execute unknown shell scripts from the internet
- run destructive commands
```

```
- deploy to production automatically
- send external customer communications automatically
- modify legal, compliance, or financial records without approval

## High-Risk File Patterns

The following files or directories are considered high-risk:

```text

.env
.env.*
secrets.*
credentials.*
config/production.*
migrations/
auth/
billing/
payments/
permissions/
security/
deploy/
infra/
```

Default Behavior

When an action is not clearly allowed, the agent must treat it as approval-required.

```
---

## 9.3 `.ai-harness/tool-policy.md`

```markdown
# Tool Governance Policy

## Purpose

This file defines how AI agents may use tools during execution.

## Read Tools

Allowed:

- read files
- search files
- inspect codebase
- inspect git diff
- inspect logs
- inspect package configuration
```

````
- inspect test files

## Write Tools

Allowed within story scope:

- edit existing implementation files
- create new implementation files
- create or update tests
- update related documentation
- update execution log
- update story state

Requires approval:

- delete files
- mass rename files
- modify configuration files
- modify CI/CD files
- modify deployment files
- modify database migrations
- modify security-sensitive files
- modify files outside story scope

## Command Tools

Allowed commands:

```bash

npm run lint
npm run typecheck
npm run test
npm run test:unit
npm run test:integration
npm run build
pnpm lint
pnpm typecheck
pnpm test
pnpm build
yarn lint
yarn test
yarn build
````

Approval-required commands:

```
npm install
pnpm add
yarn add
npm update
```
```

```
pnpm update
yarn upgrade
npm run migrate
npm run deploy
docker compose down
docker system prune
git push
```

Forbidden commands:

```
rm -rf /
rm -rf *
curl unknown-url | bash
wget unknown-url | bash
chmod 777 -R .
access production env
printenv with secrets
```

External Tools

The agent must request approval before using:

- external APIs
- production services
- cloud consoles
- payment systems
- email systems
- CRM systems
- HR systems
- analytics systems with private data

Tool Usage Logging

Every meaningful tool action must be logged in:

```
.ai-harness/execution-log.md
```

Required log fields:

- tool or command used
- purpose
- result
- failure if any
- follow-up action

```
---
```

````
## 9.4 `.ai-harness/story-state.json`

```json
{
  "story_id": "",
  "title": "",
  "status": "Draft",
  "owner_agent": "",
  "current_step": "",
  "last_action": "",
  "blockers": [],
  "next_action": "",
  "risk_level": "medium",
  "requires_human_approval": false,
  "approval_reason": "",
  "files_in_scope": [],
  "files_changed": [],
  "commands_run": [],
  "tests_status": "not_run",
  "verification_status": "not_started",
  "review_status": "not_started",
  "updated_at": ""
}
```
````

Recommended Story Status Values

```
Draft
Ready for Dev
In Progress
Code Complete
Test Running
Test Failed
Fixing
Ready for Review
Review Failed
Approved
Done
Blocked
```

9.5 `.ai-harness/verification-gates.md`

````
# Verification Gates

## Gate 1: Story Readiness

Before implementation starts, confirm:
````

```
- [ ] Story has clear acceptance criteria
- [ ] Story has technical context
- [ ] Story has bounded scope
- [ ] Story has expected output
- [ ] Story has test requirements
- [ ] Dependencies are identified
- [ ] Risk level is assigned
- [ ] Files likely to change are listed

## Gate 2: Permission Check

Before editing files, confirm:

- [ ] Files are within story scope
- [ ] No forbidden files are touched
- [ ] No approval-required action is planned without approval
- [ ] No production secrets are accessed
- [ ] No external system is modified
- [ ] Risk level is acceptable

## Gate 3: Implementation Check

After implementation, confirm:

- [ ] Changed files match story scope
- [ ] No unrelated refactor was performed
- [ ] No forbidden file was touched
- [ ] No tests were removed to make implementation pass
- [ ] No security check was bypassed
- [ ] Code follows existing project conventions

## Gate 4: Test Check

Before marking code complete, run required tests:

- [ ] Unit tests pass
- [ ] Integration tests pass if applicable
- [ ] E2E tests pass if applicable
- [ ] Lint passes
- [ ] Typecheck passes
- [ ] Build passes
- [ ] Regression risk is reviewed

## Gate 5: Review Check

Before marking story done, confirm:

- [ ] Acceptance criteria are met
- [ ] Test evidence is available
- [ ] Execution log is complete
- [ ] Failure notes are documented
```

```
- [ ] Human approval is completed if required
- [ ] Final summary is written

## Done Definition

A story can be marked as Done only when all required gates pass.
```

9.6 `.ai-harness/execution-log.md`

```
# Execution Log

## Story

Story ID:

Story Title:

Date:

Agent:

## Input Artifacts

- PRD:
- Architecture:
- Story:
- Related documents:

## Initial Understanding

Summarize the task in 3-5 bullet points.

## Scope

### Files Expected to Change

```text

[List files here]
```

Files Out of Scope

```
[List files here]
```
```

Permission Check

- Risk level:
- Approval required:
- Approval reason:
- Approval status:

Actions Taken

1. 2. 3.

Tool Usage

Tool / Command	Purpose	Result	Notes

Files Changed

```
[List changed files here]
```

Tests Run

```
[List commands here]
```

Test Results

```
[Paste summary here]
```

Failures Encountered

Failure 1

- What failed:
- Root cause:
- Fix:
- Verification after fix:

Verification Summary

- Acceptance criteria met:
- Tests passed:
- Build passed:
- Lint/typecheck passed:
- Scope respected:
- Human review required:

Final Result

```
Done / Not Done / Blocked
```

Next Action

```
[Next step]
```

````
---

## 9.7 `.ai-harness/failure-taxonomy.md`

```markdown
# Failure Taxonomy

## Purpose

This file defines common AI execution failure types and recommended
responses.

## Failure Types

### 1. Context Failure

The agent lacks necessary information.

Examples:

- missing PRD
- unclear story
- incomplete architecture
- unknown API contract

Recommended response:

- stop execution
- request missing context
- update story readiness checklist

### 2. Scope Failure

The agent modifies files outside the intended scope.

Examples:

- unrelated refactor
- touching unrelated modules
````
````

```
- changing public APIs unnecessarily

Recommended response:

- revert unrelated changes
- update execution log
- request approval if broader change is needed

### 3. Permission Failure

The agent attempts an approval-required or forbidden action.

Examples:

- modifying auth logic
- changing payment flow
- editing production config
- adding dependency without approval

Recommended response:

- stop execution
- document reason
- request human approval

### 4. Tool Failure

A tool or command fails.

Examples:

- test command fails
- build fails
- package install fails
- external API unavailable

Recommended response:

- classify failure
- retry only if safe
- document output
- fix root cause if within scope

### 5. Verification Failure

Implementation does not pass required gates.

Examples:

- tests fail
- build fails
```

```
- acceptance criteria not met
- lint/typecheck error

Recommended response:

- do not mark done
- fix issue
- rerun verification
- update execution log

### 6. Reasoning Failure

The agent misunderstands the task.

Examples:

- implements wrong feature
- ignores acceptance criteria
- makes unsupported assumptions

Recommended response:

- stop
- restate task
- compare against story
- correct implementation

### 7. Security Failure

The agent introduces or touches security-sensitive logic incorrectly.

Examples:

- weak auth check
- exposed secret
- bypassed permission check
- insecure token handling

Recommended response:

- stop immediately
- mark high risk
- require human review
- add security-specific tests

### 8. State Failure

The agent loses track of progress.

Examples:
```

```
- repeats previous step
- forgets test status
- marks task complete prematurely

Recommended response:

- update story-state.json
- reconstruct execution log
- resume from latest verified step
```

9.8 `.ai-harness/human-approval-policy.md`

```
# Human Approval Policy

## Purpose

This file defines when human approval is required.

## Approval Required For

- security-sensitive changes
- authentication changes
- authorization changes
- payment or billing changes
- database schema changes
- production configuration changes
- dependency installation
- deployment
- external communication
- large refactors
- changes outside story scope

## Approval Format

The agent must request approval using this format:

```text

Approval Required

Action:
Reason:
Risk:
Files affected:
Alternatives:
Recommended option:
```
```

Approval Result

Human owner must respond with one of:

```
Approved
Rejected
Approved with changes
Need more information
```

Approval Logging

All approvals must be recorded in:

```
.ai-harness/execution-log.md
```

Default Rule

No response means no approval.

````
---

# 10. Agent Protocols

## 10.1 `commands/dev-agent-protocol.md`

```markdown
# Dev Agent Execution Protocol

## Purpose

This protocol governs how the Dev Agent executes a story.

## Before Implementation

The Dev Agent must:

1. Read PRD.
2. Read Architecture.
3. Read current Story.
4. Read `.ai-harness/harness-manifest.md`.
5. Read `.ai-harness/permissions.md`.
6. Read `.ai-harness/tool-policy.md`.
7. Read `.ai-harness/verification-gates.md`.
8. Confirm story scope.
9. Identify likely files to change.
10. Identify required tests.
11. Update `.ai-harness/story-state.json`.
````
````

```
## During Implementation

The Dev Agent must:

1. Change only files inside story scope.
2. Avoid unrelated refactors.
3. Follow existing code conventions.
4. Add or update relevant tests.
5. Update story state after major steps.
6. Log meaningful actions.
7. Stop if permission boundary is reached.
8. Stop if required context is missing.
9. Stop if forbidden action is required.

## After Implementation

The Dev Agent must:

1. Run required verification commands.
2. Fix failed tests if within scope.
3. Document failures.
4. Update execution log.
5. Update story state.
6. Produce final verification summary.
7. Mark story as Done only if all gates pass.

## Stop Conditions

The Dev Agent must stop when:

- action requires human approval
- story scope is unclear
- required context is missing
- tests fail for reasons outside scope
- forbidden file must be changed
- production secret is required
- destructive command seems necessary
```

10.2 `commands/qa-agent-protocol.md`

```
# QA Agent Protocol

## Purpose

This protocol governs how the QA Agent verifies a completed story.

## QA Responsibilities
```

````
The QA Agent checks:

- acceptance criteria
- test coverage
- regression risk
- changed files
- permission compliance
- execution log completeness
- failure handling
- human approval status

## QA Review Steps

1. Read story.
2. Read acceptance criteria.
3. Read changed files.
4. Read execution log.
5. Confirm tests were run.
6. Confirm test results.
7. Check if files changed are within scope.
8. Check whether approval-required actions occurred.
9. Check if failures are documented.
10. Decide review result.

## QA Result Format

```text

QA Review Result: Pass / Fail / Needs Human Review

Summary:

-

Evidence:

-

Issues:

-

Required Fixes:

-

Approval Required:

- Yes / No
````
````

QA Pass Criteria

QA can pass the story only when:

- all acceptance criteria are met
- tests pass
- build passes
- no forbidden action occurred
- approval-required actions were approved
- execution log is complete
- no critical regression risk remains

````
---

## 10.3 `commands/story-readiness-check.md`

```markdown
# Story Readiness Check

## Purpose

This check ensures a story is ready for AI agent execution.

## Checklist

- [ ] Story title is clear
- [ ] Business goal is clear
- [ ] User value is clear
- [ ] Acceptance criteria are explicit
- [ ] Technical context is provided
- [ ] Relevant architecture links are included
- [ ] Dependencies are identified
- [ ] Files likely to change are listed
- [ ] Files out of scope are listed
- [ ] Test expectations are defined
- [ ] Risk level is assigned
- [ ] Human approval needs are identified

## Story Is Not Ready If

- acceptance criteria are vague
- scope is too broad
- architecture is missing
- required data is unavailable
- high-risk changes are hidden
- test expectations are missing

## Output Format

```text
````

```
Story Readiness: Ready / Not Ready

Missing Information:

-

Risks:

-

Recommended Next Step:

-
```

```
---

## 10.4 `commands/verification-gate.md`

```markdown
# Verification Gate Command

## Purpose

This command checks whether a story can be marked as complete.

## Required Inputs

- current story
- acceptance criteria
- execution log
- changed files
- test results
- story state
- permission policy

## Verification Checklist

- [ ] Acceptance criteria are satisfied
- [ ] Required tests were run
- [ ] Tests passed
- [ ] Build passed
- [ ] Lint/typecheck passed
- [ ] Changed files are within scope
- [ ] No forbidden action occurred
- [ ] Approval-required actions were approved
- [ ] Failures are documented
- [ ] Execution log is complete

## Output Format
```
```

```text

Verification Result: Pass / Fail / Needs Human Review

Passed:

-

Failed:

-

Evidence:

-

Required Action:

-
```

10.5 `commands/permission-check.md`

```markdown
# Permission Check Command

## Purpose

This command checks whether planned agent actions are allowed.

## Input

The agent must provide:

- planned action
- files affected
- commands needed
- external systems involved
- expected risk level

## Decision Rules

Return one of:

```text

Allowed
Approval Required

```
Forbidden
Need More Information
```

Output Format

```
Permission Result:

Action:
Files:
Commands:
Risk:
Decision:
Reason:
Required Approval:
```

````
---

# 11. Example: Password Reset Story

## 11.1 Story

```markdown
# Story: Password Reset Flow

## Goal

As a user, I want to reset my password using my email so that I can regain
access to my account.

## Acceptance Criteria

- User can request a password reset email.
- Reset token expires after 30 minutes.
- User can set a new password using a valid token.
- Expired token cannot be used.
- Used token cannot be reused.
- Weak passwords are rejected.
- Reset action invalidates existing sessions.

## Test Requirements

- Unit test for token generation.
- Unit test for token expiry.
- Integration test for reset request.
- Integration test for successful reset.
- Integration test for expired token.
````
````

```
- Integration test for reused token.
- Integration test for weak password rejection.
```

11.2 Harness State

```
{
  "story_id": "AUTH-001",
  "title": "Password Reset Flow",
  "status": "Ready for Dev",
  "owner_agent": "dev",
  "current_step": "permission_check",
  "last_action": "story_readiness_passed",
  "blockers": [],
  "next_action": "inspect auth module and identify files to change",
  "risk_level": "high",
  "requires_human_approval": true,
  "approval_reason": "Authentication logic is high-risk",
  "files_in_scope": [
    "src/auth/reset-token.ts",
    "src/auth/reset-password.ts",
    "tests/auth/reset-password.test.ts"
  ],
  "files_changed": [],
  "commands_run": [],
  "tests_status": "not_run",
  "verification_status": "not_started",
  "review_status": "not_started",
  "updated_at": "2026-06-11"
}
```

11.3 Permission Result

```
Permission Result:

Action:
Implement password reset flow.

Files:
- src/auth/reset-token.ts
- src/auth/reset-password.ts
- tests/auth/reset-password.test.ts

Commands:
- npm run test
- npm run typecheck
- npm run build

Risk:
```

```
High

Decision:
Approval Required

Reason:
This story modifies authentication-related logic.

Required Approval:
Human owner must approve before implementation.
```

---

12. Development Roadmap

Phase 1: Template MVP

Goal:

```
Create a copy-pasteable BMad Harness template.
```

Deliverables:

- README.md
- `.ai-harness/` templates
- agent protocols
- checklists
- one example project

Success criteria:

```
A user can copy the template into an existing BMad project and start using it
immediately.
```

Phase 2: BMad Integration Guide

Goal:

```
Explain exactly how to combine this extension with BMad Method.
```

Deliverables:

- `docs/bmad-integration.md`
- mapping between BMad agents and Harness roles

- story lifecycle integration
- BMad Dev Agent prompt extension
- QA Agent prompt extension

Success criteria:

```
A BMad user understands where to insert Harness gates into their workflow.
```

Phase 3: CLI Init Tool

Goal:

```
Allow users to initialize the harness automatically.
```

Possible commands:

```
npx bmad-harness init
```

or:

```
uvx bmad-harness init
```

CLI should create:

```
.ai-harness/
commands/
checklists/
```

Success criteria:

```
A user can install the harness template in under one minute.
```

Phase 4: Agent-Specific Adapters

Goal:

```
Support popular AI coding environments.
```

Adapters:

- Claude Code
- Cursor
- Windsurf
- GitHub Copilot
- Cline
- Roo Code

Deliverables:

```
adapters/
  claude-code/
  cursor/
  windsurf/
  copilot/
  cline/
  roo/
```

Success criteria:

```
Each adapter provides environment-specific instructions and command files.
```

Phase 5: Enterprise Governance Layer

Goal:

```
Make this useful for enterprise AI adoption.
```

Add:

- audit log format
- approval workflow
- risk scoring
- compliance checklist
- security review gate
- change management template
- release readiness checklist

Success criteria:

```
The project can be positioned as an enterprise AI coding governance
framework.
```

---

13. Recommended Initial Issues

Create these GitHub issues after repo initialization.

Issue 1: Create Core Harness Templates

Description:

```
Add core .ai-harness templates:
- harness-manifest.md
- permissions.md
- tool-policy.md
- story-state.json
- verification-gates.md
- execution-log.md
```

Labels:

```
mvp
template
```

Issue 2: Create Dev Agent Protocol

Description:

```
Create commands/dev-agent-protocol.md defining before/during/after execution
rules for AI coding agents.
```

Labels:

```
agent-protocol
mvp
```

Issue 3: Create QA Agent Protocol

Description:

```
Create commands/qa-agent-protocol.md defining story verification and review
rules.
```

Labels:

```
agent-protocol
quality
```

Issue 4: Add Example Password Reset Story

Description:

```
Create examples/password-reset showing how Harness templates apply to a high-
risk authentication story.
```

Labels:

```
example
documentation
```

Issue 5: Write BMad Integration Guide

Description:

```
Explain how BMad Method users should insert Harness gates into their existing
workflow.
```

Labels:

```
documentation
bmad
```

---

14. GitHub README Positioning

Use this short copy near the top of the README:

```
## Why This Exists

BMad Method is great at helping AI agents plan software development.

But planning is not enough.
```

```
When AI agents execute code changes, teams need:

- verification gates
- permission boundaries
- tool policies
- story state tracking
- failure attribution
- execution logs
- human approval checkpoints

This project adds that missing governance layer.

BMad helps agents plan and build.

BMad Harness helps agents execute safely, verify results, and leave auditable
traces.
```

---

15. Suggested Tagline

```
Make AI coding agents verifiable, permission-aware, and auditable.
```

Alternative:

```
The missing execution governance layer for BMad Method.
```

Alternative:

```
From AI-assisted development to AI-governed execution.
```

---

16. Naming Options

Recommended repository name:

```
bmad-harness-extension
```

Other options:

```
bmad-hos
bmad-governance-kit
```

```
bmad-execution-harness
bmad-enterprise-harness
bmad-vspot-harness
```

Best choice:

```
bmad-harness-extension
```

Reason:

- clear
- searchable
- does not replace BMad
- easy for BMad users to understand
- good GitHub SEO

---

17. License Recommendation

Use MIT License for maximum adoption.

If you want stronger open-source protection, use Apache 2.0.

Recommended:

```
MIT
```

---

18. Versioning

Use semantic versioning.

```
v0.1.0 - Markdown template MVP
v0.2.0 - BMad integration guide
v0.3.0 - Example projects
v0.4.0 - CLI init tool
v0.5.0 - Agent adapters
v1.0.0 - Stable governance framework
```

---

19. Contribution Guide Draft

````
# Contributing

Thanks for your interest in BMad Harness Extension.

## Contribution Areas

You can contribute by improving:

- harness templates
- agent protocols
- verification gates
- permission policies
- example projects
- BMad integration docs
- AI coding tool adapters

## Principles

All contributions should support the core goal:

```text

Make AI agent execution safer, more verifiable, and more auditable.
````

Pull Request Requirements

Before submitting a PR, please confirm:

- [ ] change is within project scope
- [ ] documentation is clear
- [ ] examples are updated if needed
- [ ] templates remain tool-agnostic
- [ ] permission and verification principles are preserved

````
---

# 20. Final Summary

BMad Harness Extension is not a replacement for BMad Method.

It is the missing execution governance layer.

```text
BMad Method:
Planning, roles, PRD, architecture, stories, implementation flow.
````

```
BMad Harness Extension:
Verification, state, permission, observability, tool governance, human
approval.
```

Together:

```
BMad + Harness = AI-native software development operating system.
```

This project should start as a template repository, then evolve into a CLI, then into adapters for popular AI coding environments.

The first version should stay simple:

```
Markdown templates
Agent protocols
Checklists
Examples
```

Do not overbuild.

Ship the template first.